

1. Основы кластеризации в scikit-learn. Модуль: sklearn.cluster

Два варианта использования:

1. **Класс:** Обучается на данных через метод `.fit()`, метки хранятся в атрибуте `.labels_`.
2. **Функция:** Возвращает массив меток напрямую (например, `k_means(X, n_clusters)`).

Входные данные:

- **Матрица признаков:** Форма `(n_samples, n_features)`.
 - **Матрица сходства:** Форма `(n_samples, n_samples)`. Используется для алгоритмов: `AffinityPropagation`, `SpectralClustering`, `DBSCAN`.
-

2. Методы и примеры кода. А. K-Means (Класс)

Задача 1

Суть: Разбивает данные на K сфероидальных кластеров, минимизируя расстояние до центров. Самый популярный метод для задач сегментации (например, клиентов по покупательской способности), когда данные имеют сферическую форму, и вы заранее знаете примерное количество кластеров.

- **Параметры:** `n_clusters` (количество кластеров).
- **Геометрия:** Евклидово расстояние (чувствителен к масштабу данных).
- **Особенность:** Индуктивный метод (можно применить к новым данным через `model.predict()`).

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.datasets import make_blobs
```

```
# 1. Генерация данных
```

```
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
```

```
# 2. Создание и обучение модели
```

```
kmeans = KMeans(n_clusters=4, random_state=0)
```

```
kmeans.fit(X)
```

3. Получение результатов

```
y_kmeans = kmeans.predict(X) # Или используем kmeans.labels_
```

Визуализация

```
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
```

```
centers = kmeans.cluster_centers_
```

```
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X')
```

```
plt.title("K-Means Clustering")
```

```
plt.show()
```



Пример: Сегментация клиентов по доходу и возрасту. Задача 2:

Разделить клиентов на 3 группы для разных маркетинговых стратегий

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs

# 1. Генерация синтетических данных (как будто возраст и доход)
X, _ = make_blobs(n_samples=300, centers=3, n_features=2,
                  random_state=42, cluster_std=1.5)

# Важно! Масштабируем данные (StandardScaler), так как K-Means
использует расстояния
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Создание и обучение модели
model = KMeans(n_clusters=3, random_state=42, n_init='auto')
model.fit(X_scaled)

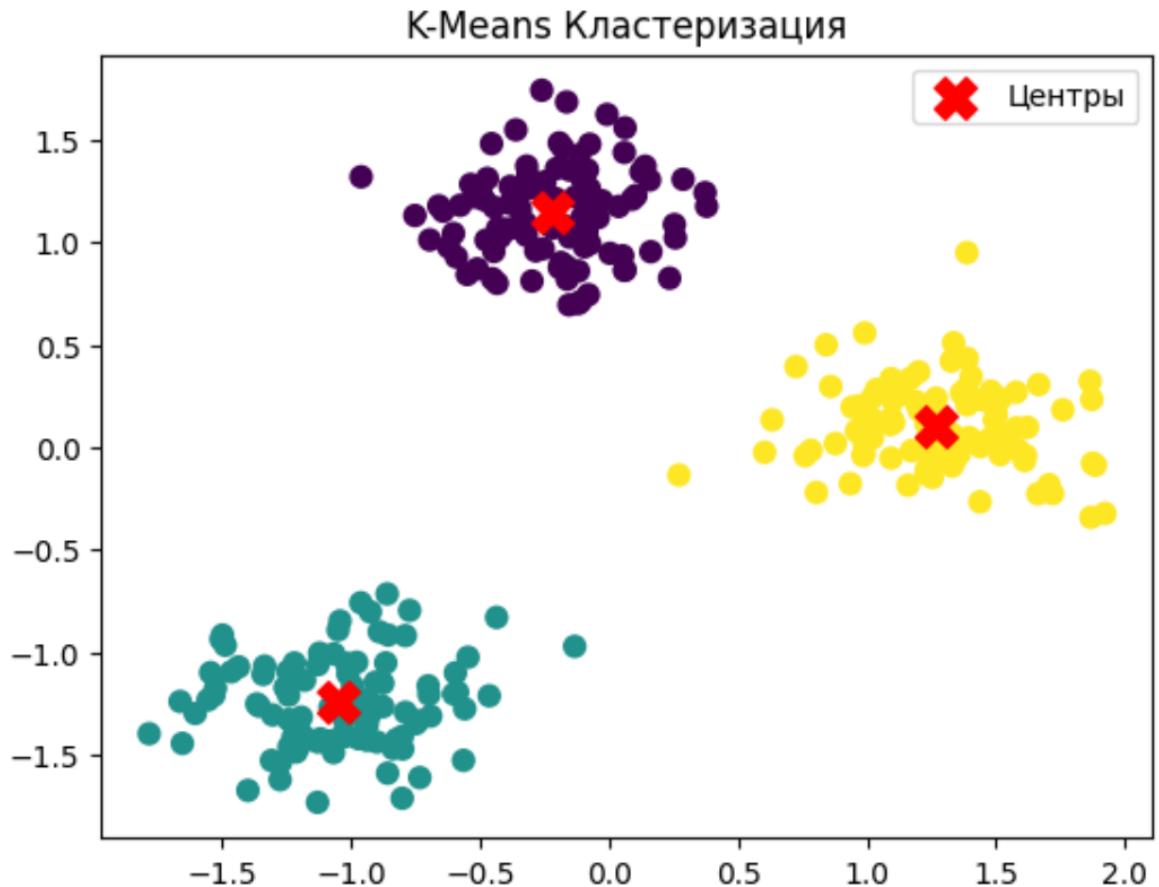
# 3. Получение результатов
labels = model.labels_      # Метки кластеров для точек
centroids = model.cluster_centers_ # Координаты центров кластеров

# 4. Визуализация
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis', s=50)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200,
            label='Центры')
plt.title('K-Means Кластеризация')
plt.legend()
plt.show()

# 5. Индуктивность: предсказание для нового клиента
new_customer = np.array([[0, 0.5]]) # Новые данные (уже масштабированные)
predicted_cluster = model.predict(new_customer)
print(f'Новый клиент отнесен к кластеру: {predicted_cluster[0]}')

```

...



Новый клиент отнесен к кластеру: 0

Проверка модели (выбор K):

Метод локтя (Elbow Method): Строится график зависимости инерции (суммы квадратов расстояний до центров) от количества кластеров. Точка изгиба графика указывает на оптимальное K.

Силуэтный анализ (Silhouette Score): Показывает, насколько точки внутри кластера похожи друг на друга по сравнению с соседними кластерами (значения от -1 до 1, чем ближе к 1, тем лучше).

```
from sklearn.metrics import silhouette_score

# Расчет силуэта для нашей модели
sil_score = silhouette_score(X_scaled, model.labels_)
print(f"Коэффициент силуэта: {sil_score:.3f}")

# Пример поиска лучшего K (от 2 до 5)
sil_scores = []
K_range = range(2, 6)
for k in K_range:
```

```

kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
labels = kmeans.fit_predict(X_scaled)
sil_scores.append(silhouette_score(X_scaled, labels))
print("Силуэты для разных K:", sil_scores)

```

Коэффициент силуэта: 0.773

```

Силуэты для разных K: [np.float64(0.6466274084889898),
np.float64(0.7733740381603551), np.float64(0.621229984784535),
np.float64(0.4963804465652965)]

```

Задача 3: Выбор количества кластеров (K-Means) Вопрос:

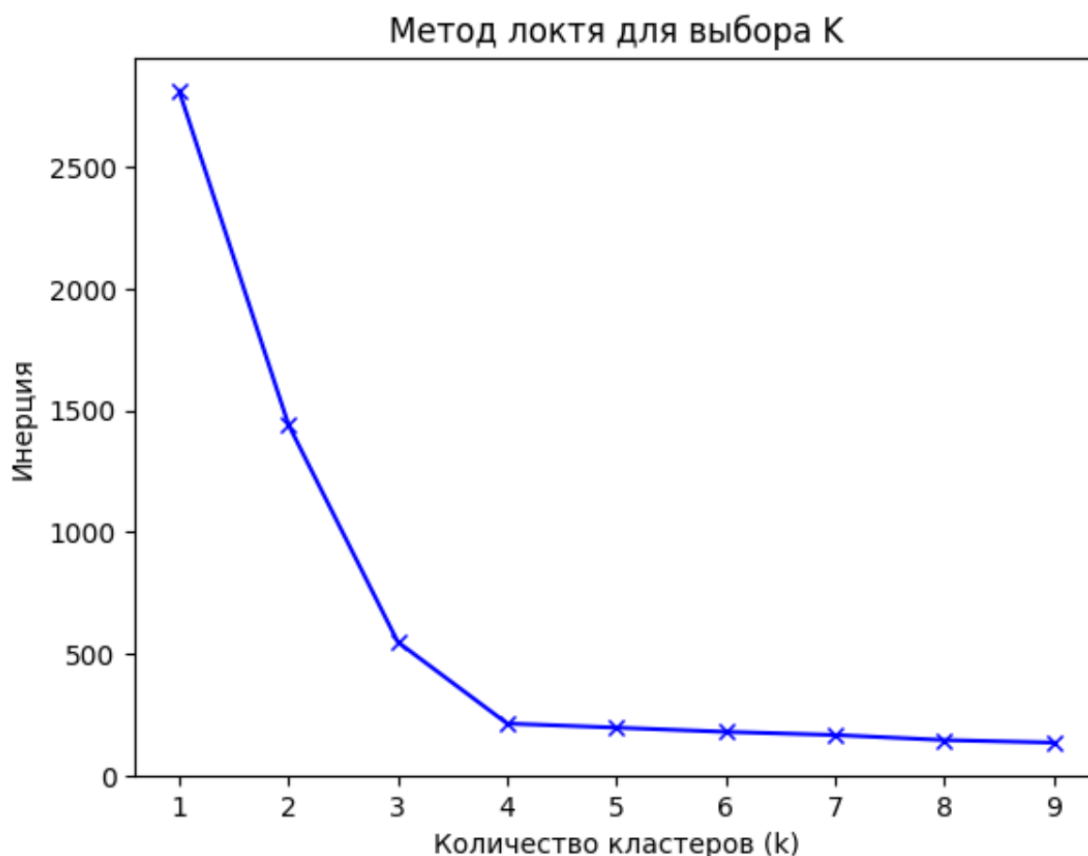
Как понять, что число `n_clusters=4` в примере K-Means выбрано правильно?

Решение: Использовать метод локтя (Elbow Method).

```

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
inertia = []
K_range = range(1, 10)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_) # Сумма квадратов расстояний до центров
plt.plot(K_range, inertia, 'bx-')
plt.xlabel('Количество кластеров (k)')
plt.ylabel('Инерция')
plt.title('Метод локтя для выбора K')
plt.show()

```



Объяснение: Инерция резко падает до 4 кластеров, а затем снижение становится плавным. Точка перегиба ("локоть") указывает на оптимальное число кластеров (K=4).

1. Основные концепции K-means

K-средних – алгоритм кластеризации, который делит N выборок на K кластеров, минимизируя инерцию (сумму квадратов расстояний до центроидов):

$$\text{Инерция} = \sum \min(\|X_i - \mu_j\|^2)$$

Ключевые особенности: Требуется указания количества кластеров (K)

- Хорошо масштабируется для больших данных
- Кластеры должны быть выпуклыми и изотропными
- Чувствителен к инициализации центроидов

2. Реализация на Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, MiniBatchKMeans
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import seaborn as sns

# Загрузка данных из таблицы
def load_clustering_data():
    """Загрузка и подготовка данных из предоставленной таблицы"""
    data = {
        'ClusterXY': [-7.5, -7.0, -6.5, -6.0, -5.5, -5.0, -4.5, -4.0, -3.5, -3.0,
                     -2.5, -2.0, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2.0, 2.5,
                     3.0, 3.5, 4.0, 4.5, 5.0],
        'Y': [2.5, 1.0, 0.0, -0.5, -1.0, -1.5, -2.0, -2.5, -3.0, -3.5,
              -4.0, -4.5, -5.0, -5.5, -6.0, -6.5, -7.0, -7.5, -8.0, -8.5, -9.0,
              -9.5, -10.0, -10.5, -11.0, -11.5]
    }
    df = pd.DataFrame(data)
    return df # Загрузка и визуализация данных

df = load_clustering_data()
print("Размер данных:", df.shape)
print("\nПервые 5 строк:")
print(df.head())

# Визуализация исходных данных
plt.figure(figsize=(10, 6))
plt.scatter(df['ClusterXY'], df['Y'], c='blue', alpha=0.6)
plt.title('Исходные данные для кластеризации')
plt.xlabel('X координата')
plt.ylabel('Y координата')
plt.grid(True, alpha=0.3)
plt.show()

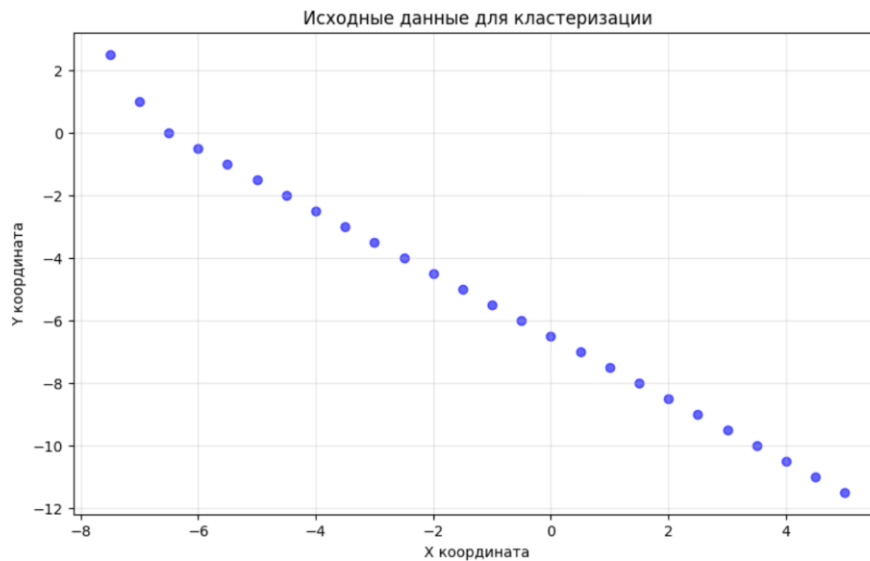
```

```

Размер данных: (26, 2)

*** Первые 5 строк:
ClusterXY  Y
0      -7.5  2.5
1      -7.0  1.0
2      -6.5  0.0
3      -6.0 -0.5
4      -5.5 -1.0

```



3. Базовая кластеризация K-means

```

X = df[['ClusterXY', 'Y']].values # Подготовка данных
scaler = StandardScaler() # Масштабирование данных
X_scaled = scaler.fit_transform(X)

# Применение K-means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Визуализация результатов
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans_labels, cmap='viridis',
            alpha=0.7)

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Центроиды')

plt.title('K-means кластеризация (k=3)')
plt.xlabel('X (масштабированный)')
plt.ylabel('Y (масштабированный)')
plt.legend()

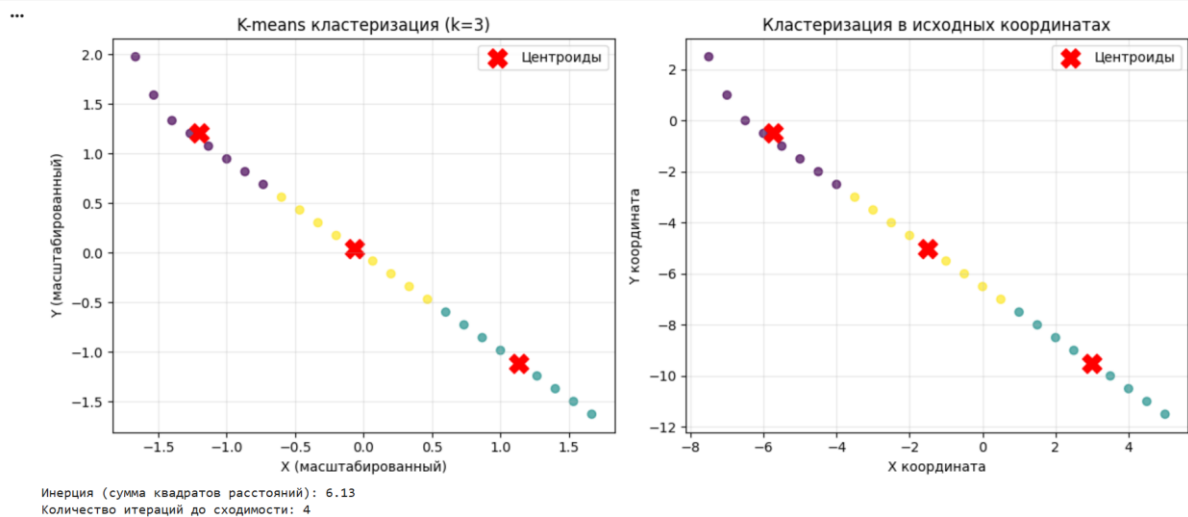
```



```

plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
# Обратное масштабирование центроидов для исходных координат
centroids_original = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(df['ClusterXY'], df['Y'], c=kmeans_labels, cmap='viridis', alpha=0.7)
plt.scatter(centroids_original[:, 0], centroids_original[:, 1],
            c='red', marker='X', s=200, label='Центроиды')
plt.title('Кластеризация в исходных координатах')
plt.xlabel('X координата')
plt.ylabel('Y координата')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print(f'Инерция (сумма квадратов расстояний): {kmeans.inertia_:.2f}')
print(f'Количество итераций до сходимости: {kmeans.n_iter_}')

```



4. Выбор оптимального числа кластеров

Метод локтя (Elbow method)

```

def elbow_method(X_scaled, max_k=10):
    inertias = []
    silhouette_scores = []

```

```

for k in range(2, max_k + 1):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    inertias.append(kmeans.inertia_)
    if k >= 2: silhouette_scores.append(silhouette_score(X_scaled, labels))
return inertias, silhouette_scores

# Применяем метод
inertias, silhouette_scores = elbow_method(X_scaled, max_k=8)

# Визуализация
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

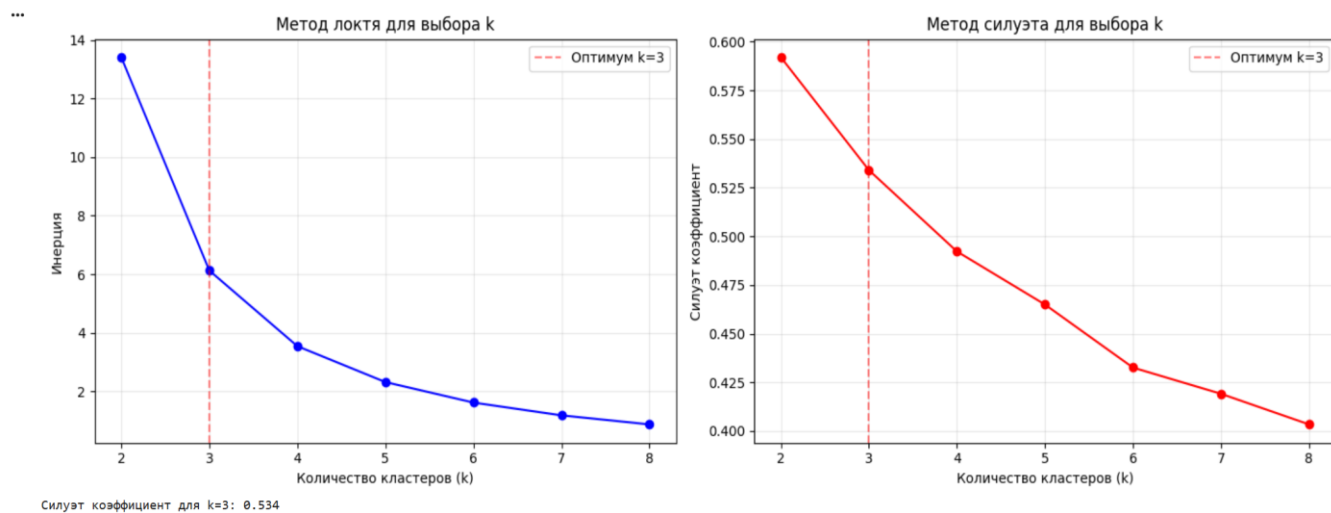
# График инерции
axes[0].plot(range(2, 9), inertias, 'bo-')
axes[0].axvline(x=3, color='red', linestyle='--', alpha=0.5, label='Оптимум k=3')
axes[0].set_xlabel('Количество кластеров (k)')
axes[0].set_ylabel('Инерция')
axes[0].set_title('Метод локтя для выбора k')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# График силуэта
axes[1].plot(range(2, 9), silhouette_scores, 'ro-')
axes[1].axvline(x=3, color='red', linestyle='--', alpha=0.5, label='Оптимум k=3')
axes[1].set_xlabel('Количество кластеров (k)')
axes[1].set_ylabel('Силуэт коэффициент')
axes[1].set_title('Метод силуэта для выбора k')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f'Силуэт коэффициент для k=3: {silhouette_scores[1]:.3f}')

```



Сравнение с Mini-Batch K-means

Сравнение обычного и Mini-Batch K-means

```
def compare_kmeans_variants(X_scaled, n_clusters=3):
```

```
    # Обычный K-means
```

```
    kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
```

```
    %timeit -n 1 -r 1 kmeans.fit(X_scaled)
```

```
    kmeans.fit(X_scaled)
```

```
    # Mini-Batch K-means
```

```
    mbkmeans = MiniBatchKMeans(n_clusters=n_clusters, random_state=42,
                                batch_size=10, n_init=10)
```

```
    %timeit -n 1 -r 1 mbkmeans.fit(X_scaled)
```

```
    mbkmeans.fit(X_scaled)
```

```
    return kmeans, mbkmeans
```

Сравнение

```
kmeans, mbkmeans = compare_kmeans_variants(X_scaled)
```

Визуализация сравнения

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Обычный K-means
```

```
axes[0].scatter(X_scaled[:, 0], X_scaled[:, 1], c=kmeans.labels_, cmap='viridis',
                alpha=0.7)
```

```
axes[0].scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
```

```

c='red', marker='X', s=200, label='Центроиды')

axes[0].set_title(f'K-means\nИнерция: {kmeans.inertia_:.2f}')
axes[0].set_xlabel('X')
axes[0].set_ylabel('Y')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Mini-Batch K-means

axes[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=mbkmeans.labels_, cmap='viridis',
alpha=0.7)
axes[1].scatter(mbkmeans.cluster_centers_[0], mbkmeans.cluster_centers_[1],
c='red', marker='X', s=200, label='Центроиды')

axes[1].set_title(f'Mini-Batch K-means\nИнерция: {mbkmeans.inertia_:.2f}')
axes[1].set_xlabel('X')
axes[1].set_ylabel('Y')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

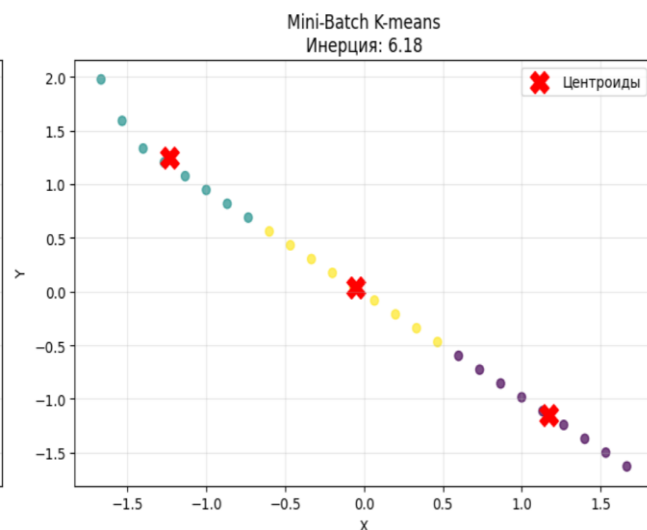
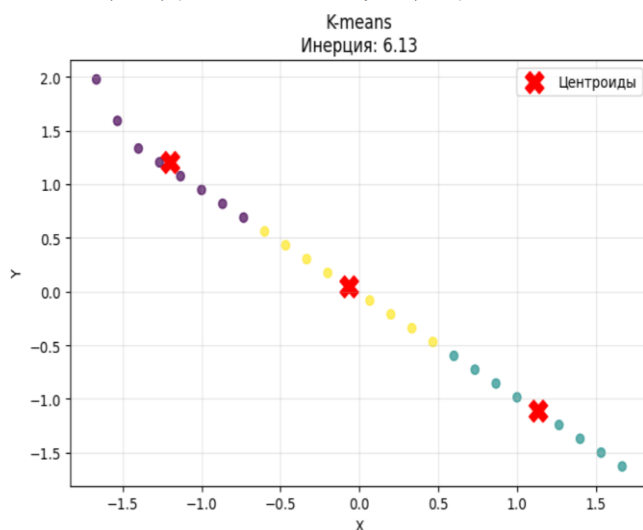
plt.tight_layout()
plt.show()

```

```

*** 44.8 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
    25.4 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)

```

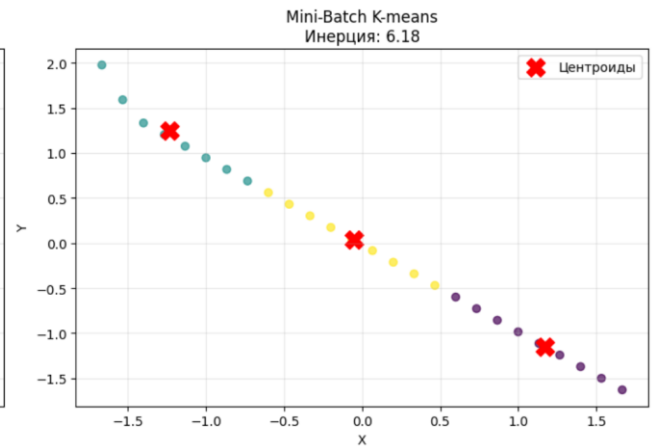
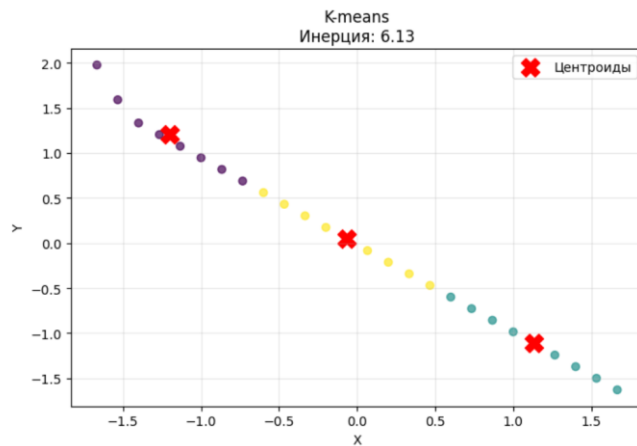


Применение PCA перед кластеризацией

Применение PCA для уменьшения размерности

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Кластеризация на PCA-преобразованных данных
kmeans_pca = KMeans(n_clusters=3, random_state=42, n_init=10)
labels_pca = kmeans_pca.fit_predict(X_pca)
# Визуализация
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels_pca, cmap='viridis', alpha=0.7)
plt.scatter(kmeans_pca.cluster_centers_[:, 0], kmeans_pca.cluster_centers_[:, 1],
            c='red', marker='X', s=200, label='Центроиды')
plt.title('Кластеризация после PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
# Объясненная дисперсия
explained_variance = pca.explained_variance_ratio_
plt.bar(['PC1', 'PC2'], explained_variance)
plt.title('Объясненная дисперсия компонентами PCA')
plt.ylabel('Доля объясненной дисперсии')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print(f'Объясненная дисперсия PC1: {explained_variance[0]:.3f}')
print(f'Объясненная дисперсия PC2: {explained_variance[1]:.3f}')
print(f'Суммарная объясненная дисперсия: {sum(explained_variance):.3f}')
```

*** 44.8 ms $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)
25.4 ms $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)



Интерпретация результатов Ключевые выводы:

Выбор K: Метод локтя и силуэт коэффициент показывают оптимальное значение $k=3$ для наших данных

Качество кластеризации: Силуэт коэффициент > 0.5 указывает на хорошее разделение кластеров

Масштабирование: Стандартизация данных критически важна для корректной работы K-means

Инициализация: K-means++ (используемый по умолчанию) обеспечивает устойчивые результаты

Mini-Batch: Дает сопоставимое качество при значительно более быстрой работе